Robot Arm Teleoperation via Smartphone Pose Estimation

Firdavs Nasriddinov & Sebastian Banuelos
ME/CDS/EE 235a: Advanced Robotics Kinematics
Professor Joel Burdick
California Institute of Technology

March 21, 2026

Demo Video: Drive
Code: GitHub

1 Motivation

The field of modern robotics has achieved unprecedented success in dexterous manipulation, largely driven by breakthroughs in imitation learning, particularly the development of Vision-Language-Action (VLA) models. These architectures have demonstrated remarkable capabilities in semantic reasoning, zero-shot generalization, and the execution of complex tasks in unstructured environments. However, the performance of these models is fundamentally constrained by the availability of high-quality human demonstration data. To capture smooth, human-like motions, researchers have to teleoperate physical systems using hardware interfaces that often bottleneck the data collection process.

Teleoperation of robot manipulators traditionally relies on teach pendants, joysticks, or spacemouse devices that map low-dimensional human input to the robot's joint or task space. These interfaces are unintuitive for 6-DOF control: operators must mentally decompose desired end-effector motions into separate translational and rotational commands, often through menus or mode switches. This disconnect between the operator's spatial intent and the input device limits both the speed and precision of teleoperation.

Modern smartphones present a compelling alternative. Equipped with inertial measurement units (IMU), cameras, and in some models LiDAR sensors, devices such as the iPhone can estimate their own 6-DOF pose in real time via visual-inertial odometry (VIO). Since humans naturally manipulate phones in 3D space—translating, rotating, and tilting them with high dexterity—the phone itself becomes an intuitive spatial controller that directly maps to the robot's task space.

2 Project Overview

In this project, we develop a teleoperation system for the UR10e 6-DOF manipulator using an iPhone as the input device. The phone's pose, tracked by Apple's ARKit framework, drives the robot through two control modes:

- **Velocity mode**: The phone's instantaneous twist (linear and angular velocity) is mapped to joint velocities via resolved-rate control using the geometric Jacobian with adaptive damped least-squares inversion.
- **Position mode**: The phone's relative displacement from a reference pose is mapped to a target end-effector pose, which is reached via analytical closed-form inverse kinematics.

The system operates in simulation, with the robot visualized in RViz 2. Real-time feedback—including manipulability, joint limit proximity, and feasibility—is computed by the kinematics node and sent back to the phone, where it is displayed through visual indicators and haptic feedback.

The project builds directly on the kinematic foundations developed throughout ME/CD-S/EE 235a: Denavit-Hartenberg parameterization, forward kinematics, the geometric Jacobian, resolved-rate control, and analytical inverse kinematics for the UR10e. The contribution of this work is the integration of these techniques into a real-time teleoperation pipeline

driven by smartphone pose estimation.

3 Implementation Details

3.1 Pose Estimation via Visual-Inertial Odometry

The iPhone's pose is estimated by Apple's ARKit framework [1] using **visual-inertial odometry (VIO)**, an algorithm closely related to the Multi-State Constraint Kalman Filter (MSCKF) [3, 2]. VIO fuses two complementary sensor systems to produce a 6-DOF pose estimate at approximately 60 Hz.

3.1.1 Inertial Odometry

The phone's IMU provides accelerometer readings $\mathbf{a}_b$ and gyroscope readings $\boldsymbol{\omega}_b$ in the body frame at high frequency (∼100–1000 Hz). The IMU state is propagated by integration:

$$\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \boldsymbol{\omega}_b \tag{1}$$

$$\dot{\mathbf{v}} = \mathbf{R}(\mathbf{q})\,\mathbf{a}_b - \mathbf{g} \tag{2}$$

$$\dot{\mathbf{p}} = \mathbf{v} \tag{3}$$

where $\mathbf{q}$ is the orientation quaternion, $\mathbf{R}(\mathbf{q})$ is the corresponding rotation matrix, $\mathbf{g}$ is gravity, and $\mathbf{v}$, $\mathbf{p}$ are velocity and position in the world frame. IMU measurements are corrupted by bias terms $\mathbf{b}_a$, $\mathbf{b}_g$ that drift slowly over time. The double integration of noisy accelerometer data causes position drift that grows quadratically, making inertial odometry alone insufficient for sustained tracking.

3.1.2 Visual Odometry

At each camera frame (∼30–60 Hz), visual features (corners, edges) are extracted and tracked across consecutive frames using descriptor matching. These feature correspondences provide geometric constraints between camera poses via epipolar geometry. While visual odometry is more accurate over long time horizons than inertial integration, it is computationally slower and sensitive to motion blur and textureless environments.

3.1.3 Sensor Fusion via Error-State EKF

The MSCKF-style filter maintains a state vector containing:

$$\mathbf{x} = \begin{bmatrix}\mathbf{q} & \mathbf{b}_g & \mathbf{v} & \mathbf{b}_a & \mathbf{p} & \mathbf{q}_{C_1} & \mathbf{p}_{C_1} & \cdots & \mathbf{q}_{C_N} & \mathbf{p}_{C_N}\end{bmatrix}^\top \tag{4}$$

where $(\mathbf{q}_{C_i}, \mathbf{p}_{C_i})$ is a sliding window of N recent camera poses. Crucially, *feature positions are not included in the state vector*. Instead, when a feature is tracked across multiple frames, its observations impose probabilistic constraints between the camera poses in the sliding window. This makes the filter computationally efficient, as the state vector size depends only on the window length, not the number of features.

The filter operates in two steps:

1. **Propagation**: IMU measurements predict the state forward between camera frames using Equations (1)–(3).
2. **Update**: When features are observed across multiple frames, their measurements correct the predicted state via an EKF update step, compensating for IMU drift.

On devices equipped with LiDAR (iPhone Pro, iPad Pro), depth measurements at 256×192 resolution resolve the scale ambiguity inherent in monocular vision and improve accuracy along the depth axis.

3.1.4 Twist Computation

ARKit outputs the camera pose as a 4×4 homogeneous transformation $\mathbf{T}_k \in SE(3)$ at each frame. Our system computes the twist (body velocity) from consecutive poses:

$$\mathbf{v} = \frac{\mathbf{p}_k - \mathbf{p}_{k-1}}{\Delta t}, \qquad \boldsymbol{\omega} = \frac{\theta\,\hat{\mathbf{n}}}{\Delta t} \tag{5}$$

where θ and $\hat{\mathbf{n}}$ are the angle and axis of the relative rotation $\mathbf{R}_k\mathbf{R}_{k-1}^\top$, extracted via the quaternion representation.

3.2 Coordinate Frame Mapping

A significant challenge in this project was mapping between the ARKit coordinate frame and the robot's coordinate frame.

3.2.1 ARKit World Frame

ARKit's world tracking uses gravity alignment: the Y-axis points up (aligned with gravity), while X and Z are horizontal, with their orientation determined by the device's heading at session initialization. This world frame is fixed for the duration of the AR session.

3.2.2 Reference-Relative Twist

The twist computed in Section 2.1 is expressed in the ARKit world frame. However, for intuitive teleoperation, movements should be relative to the phone's orientation at the time of reset or session start. For example, "move right" should always correspond to the same robot axis regardless of which compass direction the phone faces.

We solve this by rotating the twist into the reference pose's local frame before axis remapping:

$$\begin{bmatrix} \mathbf{v}_{\text{local}} \\ \boldsymbol{\omega}_{\text{local}} \end{bmatrix} = \begin{bmatrix} \mathbf{R}_{\text{ref}}^\top & \mathbf{0} \\ \mathbf{0} & \mathbf{R}_{\text{ref}}^\top \end{bmatrix} \begin{bmatrix} \mathbf{v}_{\text{world}} \\ \boldsymbol{\omega}_{\text{world}} \end{bmatrix} \tag{6}$$

where $\mathbf{R}_{\text{ref}}$ is the rotation component of the reference pose captured at reset. Since $\mathbf{R}_{\text{ref}}$ is orthogonal, $\mathbf{R}_{\text{ref}}^\top = \mathbf{R}_{\text{ref}}^{-1}$.

3.2.3 Axis Remapping

The phone and robot use different coordinate conventions. We define a configurable permutation-and-sign mapping via a matrix $\mathbf{P}$ that maps ARKit axes to robot axes:

$$\mathbf{v}_{\text{robot}} = \mathbf{P}\,\mathbf{v}_{\text{local}}, \qquad \mathbf{R}_{\text{robot}} = \mathbf{P}\,\mathbf{R}_{\text{local}}\,\mathbf{P}^{\top} \tag{7}$$

The default mapping is: robot $X \leftarrow$ phone $-Z$, robot $Y \leftarrow$ phone $-X$, robot $Z \leftarrow$ phone $+Y$.

3.2.4 Relative Transform for Position Mode

In position mode, the relative displacement from the reference pose is computed as:

$$\mathbf{T}_{\text{rel}} = \mathbf{T}_{\text{ref}}^{-1}\,\mathbf{T}_{\text{current}} \tag{8}$$

This transform is expressed in the reference frame's coordinates, then remapped to the robot frame and applied to the reference end-effector pose:

$$\mathbf{T}_{\text{target}} = \mathbf{T}_{\text{ee,ref}}\,\mathbf{T}_{\text{rel,robot}} \tag{9}$$

3.3 Velocity Mode: Resolved-Rate Control

In velocity mode, the phone's twist is mapped to joint velocities via the geometric Jacobian.

3.3.1 Forward Kinematics and Jacobian

The UR10e is parameterized using the standard Denavit-Hartenberg convention [4]. Each joint's homogeneous transform is:

$$\mathbf{A}_i = \begin{bmatrix} c_{\theta_i} & -s_{\theta_i}c_{\alpha_i} & s_{\theta_i}s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i}c_{\alpha_i} & -c_{\theta_i}s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{10}$$

The DH parameters for the UR10e are given in Table 1.

Table 1: UR10e DH Parameters (standard convention).

Joint	a_i [m]	d_i [m]	α_i [rad]
1	0	0.1807	$\pi/2$
2	-0.6127	0	0
3	-0.57155	0	0
4	0	0.17415	$\pi/2$
5	0	0.11985	$-\pi/2$
6	0	0.11655	0

The forward kinematics chain gives $\mathbf{T}_0^i = \mathbf{A}_1 \cdots \mathbf{A}_i$ for each frame. The 6×6 geometric Jacobian is constructed column-wise:

$$\mathbf{J}_i = \begin{bmatrix} \hat{\mathbf{z}}_{i-1} \times (\mathbf{o}_6 - \mathbf{o}_{i-1}) \\ \hat{\mathbf{z}}_{i-1} \end{bmatrix}, \quad i = 1, \ldots, 6 \tag{11}$$

where $\hat{\mathbf{z}}_{i-1}$ is the joint axis and $\mathbf{o}_{i-1}$ is the origin of frame $i-1$.

3.3.2 Damped Least-Squares Inverse

The joint velocities are computed via the damped least-squares (DLS) pseudoinverse:

$$\dot{\mathbf{q}} = \mathbf{J}^\top \left(\mathbf{J}\mathbf{J}^\top + \lambda^2 \mathbf{I}\right)^{-1} \dot{\mathbf{x}} \tag{12}$$

where $\dot{\mathbf{x}} = [\mathbf{v}^\top, \boldsymbol{\omega}^\top]^\top$ is the desired task-space twist and λ is a damping factor.

3.3.3 Adaptive Damping

The damping factor adapts based on the Yoshikawa manipulability measure [6]:

$$\mu = \sqrt{\det(\mathbf{J}\mathbf{J}^\top)} \tag{13}$$

As $\mu \to 0$ (approaching a singularity), λ increases to prevent large joint velocities:

$$\lambda(\mu) = \begin{cases} \lambda_{\min} & \text{if } \mu > \mu_{\text{thresh}} \\ \lambda_{\min} + \left(1 - \frac{\mu}{\mu_{\text{thresh}}}\right)(\lambda_{\max} - \lambda_{\min}) & \text{otherwise} \end{cases} \tag{14}$$

with $\lambda_{\min} = 0.001$, $\lambda_{\max} = 0.1$, and $\mu_{\text{thresh}} = 0.01$.

3.3.4 Twist Filtering

The twist from ARKit is computed via finite differencing of consecutive poses, which produces noisy velocity estimates. To smooth the input before it enters the resolved-rate solver, we apply an exponential moving average (EMA) filter:

$$\dot{\mathbf{x}}_k = \alpha\, \dot{\mathbf{x}}_{\text{raw}} + (1 - \alpha)\, \dot{\mathbf{x}}_{k-1} \tag{15}$$

with $\alpha = 0.3$. This attenuates high-frequency jitter while preserving the operator's intended motion, at the cost of a slight increase in effective latency.

3.3.5 Integration and Limits

Joint positions are updated via Euler integration:

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \dot{\mathbf{q}}_k\, \Delta t \tag{16}$$

Joint velocities are clamped to $\pm\pi$ rad/s (joints 1–3) and $\pm 2\pi$ rad/s (joints 4–6), and joint positions are clamped to $[-2\pi, 2\pi]$.

3.4 Position Mode: Analytical Inverse Kinematics

In position mode, the target end-effector pose is reached via analytical closed-form inverse kinematics, adapted from Lab 2 of this course.

3.4.1 Solution Method

Given a target pose $\mathbf{T}_{\text{target}} \in SE(3)$, we solve for the joint angles $\theta_1, \ldots, \theta_6$ sequentially using the UR10e's geometric structure:

1. θ_1: Solved from the target position's x-y components using the half-angle substitution $t = \tan(\theta/2)$, yielding up to 2 solutions from a quadratic in t.
2. θ_5, θ_6: Solved from the target orientation matrix entries that depend on θ_1.
3. θ_2: Solved via another half-angle substitution using the remaining position constraints, yielding up to 2 additional solutions per θ_1.
4. θ_3, θ_4: Solved in closed form from the remaining geometric constraints.

This yields up to 8 solutions per target pose. Each solution undergoes a safety check: the forward kinematics chain is evaluated, and any solution where an intermediate frame falls below the ground plane ($z < 0$) is discarded.

3.4.2 Solution Selection

Among the valid solutions, we prefer *elbow-up* configurations. Elbow-up configurations reduce the risk of intermediate joint frames passing through the ground plane, which is both physically impossible and a safety hazard on real hardware. For UR-series robots, the elbow-up vs. elbow-down configuration is determined by the sign of the elbow joint angle [5]: a solution is classified as elbow-up if

$$\theta_3 > 0 \tag{17}$$

Among elbow-up solutions, we select the one closest to the current joint configuration in the wrapped angular distance sense:

$$d(\mathbf{q}_{\text{sol}}, \mathbf{q}_{\text{current}}) = \|\text{atan2}(\sin(\mathbf{q}_{\text{sol}} - \mathbf{q}_{\text{current}}),\, \cos(\mathbf{q}_{\text{sol}} - \mathbf{q}_{\text{current}}))\| \tag{18}$$

If no elbow-up solutions exist, we fall back to the closest solution overall.

3.5 System Architecture

The system consists of three components communicating over WebSocket:

1. **iOS App** (Swift/SwiftUI): The `ARSessionManager` captures ARKit poses and computes twist. The `TeleopManager` coordinates AR tracking, reference pose management, frame rotation, and axis remapping. The `WebSocketManager` handles bidirectional communication with the server.

2. **FastAPI Relay Server** (Python): A lightweight WebSocket relay with no kinematics computation. It forwards teleop messages from the iOS app to the ROS node and relays feedback in the reverse direction. It also hosts a web-based debug dashboard with live charts.
3. **ROS 2 Kinematics Node** (Python, ROS 2 Jazzy): Runs a 30 Hz timer callback that performs either resolved-rate or analytical IK depending on the control mode. Publishes `JointState` messages for `robot_state_publisher` and RViz. Communicates with the server via a background WebSocket thread.

All messages use JSON encoding. The teleop message contains the twist vector, the 4×4 relative transform (row-major), the control mode, and an optional command field (e.g., "reset").

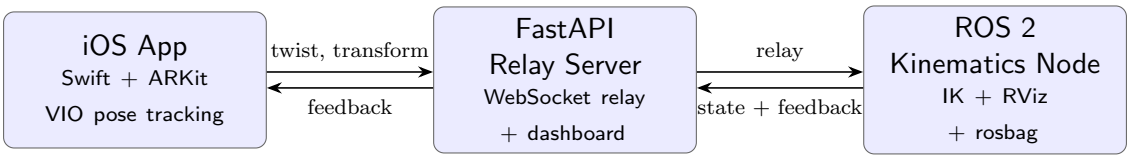

Figure 1: System architecture. The iOS app streams pose data over WebSocket to a relay server, which forwards it to the ROS 2 kinematics node. Feedback flows in the reverse direction.

3.6 Feedback System

The kinematics node computes and sends feedback to the iOS app at 30 Hz:

- **Manipulability** μ: Yoshikawa measure (Equation 13), indicating proximity to singularities.
- **Joint limit proximity**: Per-joint normalized distance to the nearest limit, where $0 =$ center of range and $1 =$ at limit.
- **Workspace proximity**: End-effector distance as a fraction of the UR10e's maximum reach ($\sim$1.18 m).
- **Feasibility**: Boolean flag, true when $\mu > 0.005$ and all joints are within limits.

The iOS app renders this as color-coded indicators (green/yellow/red) and maps manipulability to haptic vibration intensity, providing the operator with tactile awareness of the robot's kinematic state.

4 Outputs and Results

The complete system was implemented and tested in simulation. A demonstration video and all source code are linked on the title page. Below we present the key outputs.

4.1 System Demonstration

The teleoperation pipeline runs in real time: the operator holds the iPhone, starts the AR session, and moves the phone in 3D space. The UR10e model in RViz responds immediately, with end-to-end latency under 100 ms. Figure 2 shows the iOS app and RViz side by side during a teleoperation session.

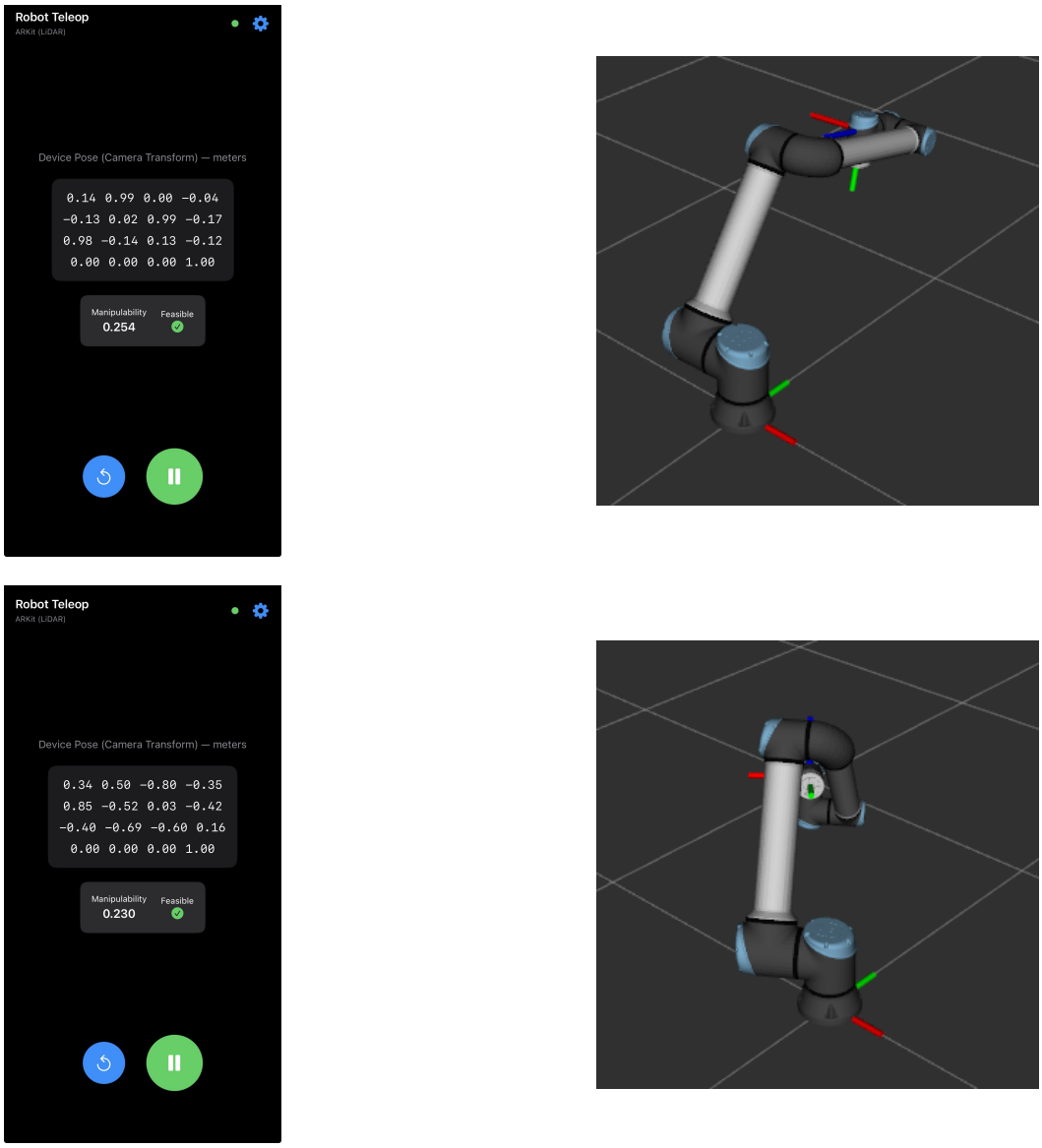

Figure 2: System in operation. Left: iOS app showing the device pose matrix and feedback indicators. Right: RViz displaying the UR10e at the corresponding joint configuration.

4.2 Position Mode

In position mode, the phone's relative displacement maps directly to a target end-effector pose via analytical IK. Figure 3 shows joint trajectories during a position-mode session.

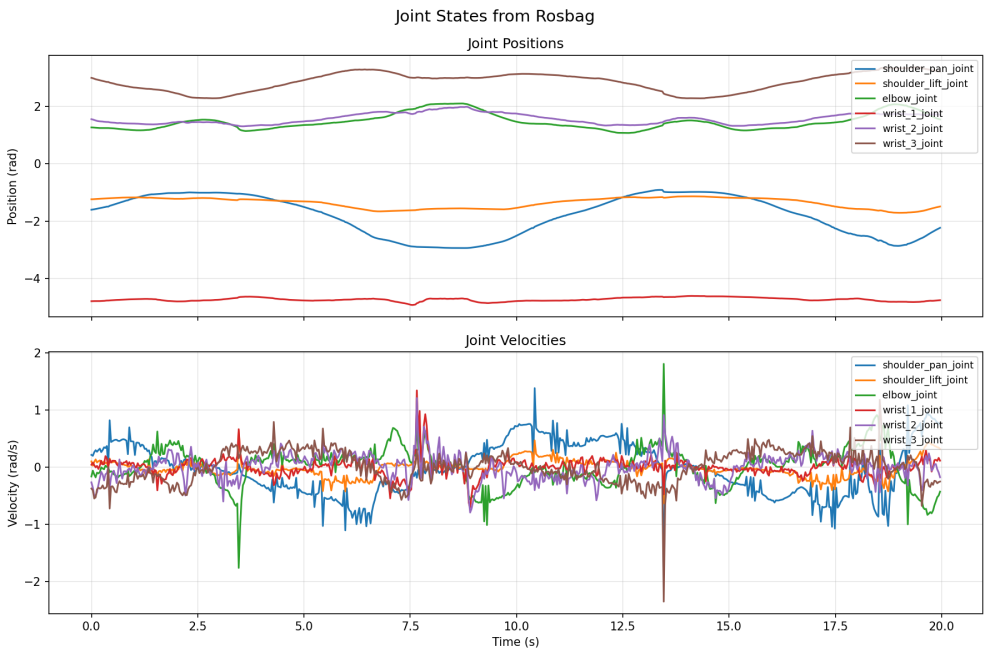

Figure 3: Joint states during a position-mode teleoperation session.

In position mode, the elbow-up solution selection ensures continuous, physically plausible joint trajectories without configuration flips. However, the velocity is not smooth and there can still be sudden discontinuous jumps in joint position as can be seen around the 13s mark in Figure 3.

4.3 Velocity Mode

In velocity mode, the phone's twist drives the robot via resolved-rate control. Figure 4 shows recorded joint positions and velocities during a velocity-mode session, captured via automatic rosbag recording. This maneuver was done to be similar of that of the position mode maneuever.

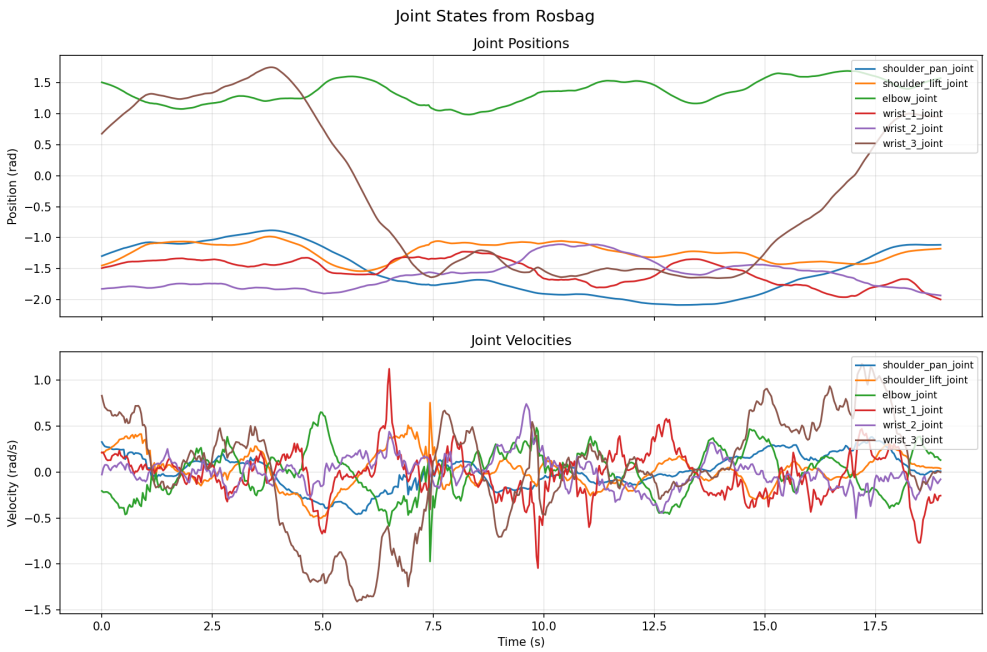

Figure 4: Joint states during a velocity-mode teleoperation session.

The smooth velocity profiles demonstrate the damped least-squares controller's singularity avoidance as well as the twist filtering. However, in velocity mode the robot's position tends to drift after doing a maneuver and then coming back to the starting position. This was observed with or without the twist filtering.

4.4 Reset Behavior

Pressing the reset button on the iOS app sends a command that snaps the robot to the home configuration $\mathbf{q}_{\text{home}} = [-\pi/2,\ -\pi/2,\ \pi/2,\ -\pi/2,\ -\pi/2,\ 0]$ and re-references the phone's current pose. Figure 5 shows the joint positions before and after a reset event. When deploying this system to a physical robot, this feature would compute a smooth trajectory from the current state to the home state, to avoid unsafe jumps in the commanded joint positions.

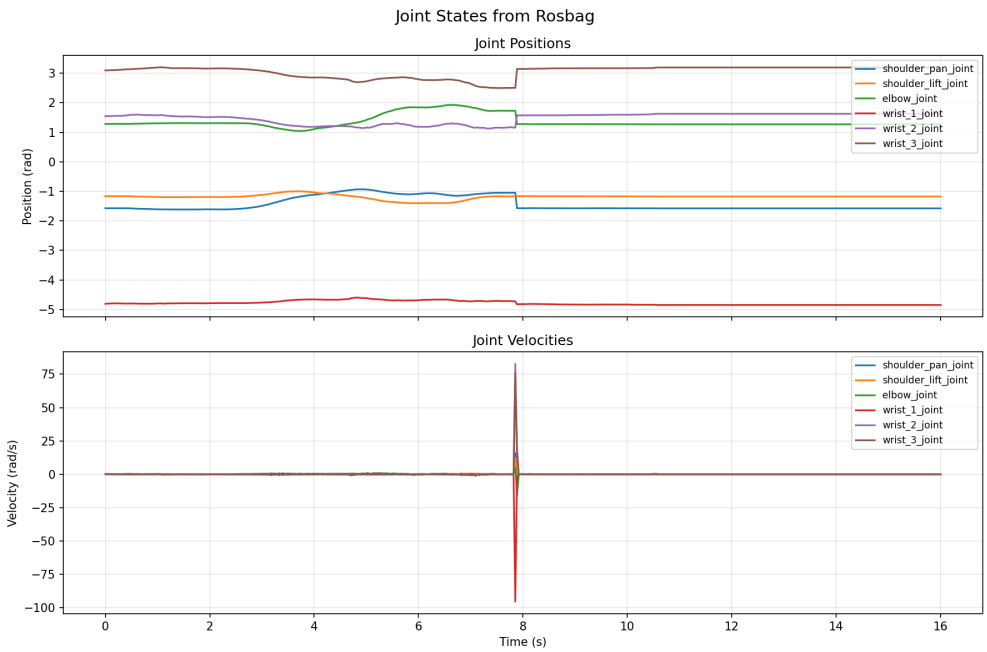

Figure 5: Reset event. The robot snaps to the home configuration, and subsequent phone movements are relative to the new reference pose.

4.5 Debug Dashboard

A web-based debug dashboard provides real-time visualization of all system telemetry, including joint angles, manipulability, twist magnitude, and joint limit proximity. Figure 6 shows the dashboard during operation. We used this while in development to refine our system.

Figure 6: Web debug dashboard showing live telemetry: joint state table, end-effector position, manipulability bar, and time-series charts for joint angles, manipulability, twist magnitude, and joint limit proximity.

5 Conclusion

We developed a real-time teleoperation system for the UR10e manipulator using an iPhone as a 6-DOF spatial input device. The system integrates smartphone pose estimation via visual-inertial odometry, coordinate frame mapping between the phone and robot, and dual-mode inverse kinematics—resolved-rate Jacobian control for velocity mode and analytical closed-form IK for position mode. Real-time feedback on manipulability, joint limits, and feasibility is communicated back to the operator.

5.1 Shortcomings

- **Simulation only**: The system was tested exclusively in RViz and was not connected to a physical UR10e. Real hardware would introduce additional challenges including communication latency, motor dynamics, and safety constraints.
- **No collision avoidance**: The system enforces joint limits but does not check for self-collisions or collisions with the environment. Operating near the workspace boundary relies entirely on the feasibility feedback to warn the operator.
- **VIO drift**: ARKit's visual-inertial odometry accumulates drift over long sessions, particularly in featureless environments. This can cause a gradual divergence between the operator's intended and actual robot motion.
- **Communication latency**: The WebSocket relay chain (phone → server → ROS)

introduces latency that, while typically under 100 ms, can affect the responsiveness of position-mode control.

- **Limited trajectory smoothing**: An exponential moving average filter is applied to the incoming twist, but no smoothing is applied to the IK output in position mode, which can produce jerky motion when the analytical IK switches between nearby solution branches.

5.2 Future Work

- **Real robot integration**: Connecting to the physical UR10e via the UR RTDE interface or `ros2_control` would validate the system in a real-world setting and enable evaluation of tracking accuracy and operator performance.
- **Grasping and manipulation**: Adding a gripper controlled by a phone gesture (e.g., pinch) would extend the system to pick-and-place tasks.
- **Further trajectory smoothing**: Extending the current twist filtering to the position-mode IK output, or applying a minimum-jerk trajectory planner, would produce smoother robot motion across both control modes.
- **Collision avoidance**: Incorporating the URDF collision meshes into the control loop would enable real-time self-collision and environment collision checks.
- **Multi-user teleoperation**: Supporting multiple phones controlling different aspects of the robot (e.g., one for position, one for orientation) could enable more complex manipulation tasks.

References

[1] Apple Inc. Understanding world tracking. `https://developer.apple.com/documentation/arkit/understanding-world-tracking`, 2024.

[2] Mingyang Li and Anastasios I. Mourikis. High-precision, consistent EKF-based visual-inertial odometry. *The International Journal of Robotics Research*, 32(6):690–711, 2013.

[3] Anastasios I. Mourikis and Stergios I. Roumeliotis. A multi-state constraint kalman filter for vision-aided inertial navigation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 3565–3572, 2007.

[4] Richard M. Murray, Zexiang Li, and S. Shankar Sastry. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.

[5] Robert L. Williams II. Universal robot URe-series cobot kinematics & dynamics. Technical report, Ohio University, Department of Mechanical Engineering, 2023.

[6] Tsuneo Yoshikawa. Manipulability of robotic mechanisms. *The International Journal of Robotics Research*, 4(2):3–9, 1985.